Structural Compression and Optimization of Large Language Models for Resource-Constrained Edge Platforms

Kishore Rajendran
ECE, UC San Diego
kirajendran@ucsd.edu

Ayush Jamdar
ECE, UC San Diego
ajamdar@ucsd.edu

Arnav Saraf
ECE, UC San Diego
arsaraf@ucsd.edu

Abstract

The deployment of Large Language Models (LLMs) on edge devices is fundamentally constrained by memory and compute requirements; even moderately sized models require hardware far beyond consumer capabilities. This work studies structural compression techniques, specifically Post Training Quantization (PTQ) and structured pruning techniques, as mechanisms to navigate the tradeoff between model size, latency, and reasoning performance.

We survey SoTA methods, including weight only quantization (GPTQ, AWQ) and activation aware approaches (SmoothQuant), with particular emphasis on outlier suppression in activation distributions. Experimentally, we evaluate 4 bit and 8 bit quantization schemes, extending beyond perplexity to downstream reasoning tasks. A module wise sensitivity analysis also reveals the relative robustness of architectural components (MLP vs. attention) to quantization noise. Our results are consolidated into accuracy memory bandwidth tradeoffs, identifying practical quantization techniques and operating points for edge deployment of LLMs of varying sizes. Our code is available at https://github.com/AyushJam/structured-compression.

1 Introduction

1.1 Fundamentals of Model Compression

State of the art (SoTA) LLMs achieve remarkable performance but are difficult to deploy on edge devices due to memory and compute constraints. For example, a 7B parameter model in FP16 requires ∼14GB for storage and up to 80GB GPU VRAM for training, exceeding consumer hardware capacity. Model compression techniques such as quantization and pruning reduce memory and computational cost while preserving predictive performance, but introduce approximation errors that trade off efficiency against accuracy, which is a critical consideration for resource constrained deployment.

1.2 Quantization

Quantization reduces memory and computational cost by mapping high precision weights (FP32/FP16) to lower precision formats (INT8, INT4, FP8). This improves efficiency but introduces approximation errors, especially due to outliers (rare extreme values that distort scaling and reduce precision).

Two main approaches exist: Quantization Aware Training (QAT) simulates quantization during training, allowing weights to adapt and maintain high accuracy even at very low bit widths (e.g. 2 bit), but it is computationally intensive and requires original training data. Post Training Quantization (PTQ), in contrast, operates on pretrained weights using a small calibration dataset (typically 128–256 samples) to determine optimal scales. PTQ is widely used for large scale open source models due to its efficiency. This work focuses primarily on PTQ.

1.3 PTQ Methodologies

1.3.1 Round to Nearest (RTN)

Round to Nearest (RTN) is a simple post training quantization baseline that independently maps each weight to the nearest discrete level:

$$\hat{W} = \text{clip}\left(\text{round}\left(\frac{W}{s}\right), q_{\min}, q_{\max}\right) \cdot s, \quad (1)$$

where s is the scale and $(q_{\min}, q_{\max})$ define the quantization range. While it minimizes element wise error, RTN ignores inter-weight dependencies and layer sensitivity, making it prone to outliers that inflate scaling, reduce precision, and degrade performance at low bit widths.

1.3.2 GPTQ: Hessian-Aware Quantization

GPTQ (Frantar et al., 2023) improves upon RTN by incorporating second-order information to min-

imize output reconstruction error. For a weight matrix $W \in \mathbb{R}^{d\times k}$, it solves:

$$\min_{\hat{W}} \|WX - \hat{W}X\|_2^2 \tag{2}$$

where X denotes input activations. Using a second-order approximation, this reduces to:

$$\min \hat{W}(W-\hat{W})^T H(W-\hat{W}), \quad H = XX^T, \tag{3}$$

where H approximates the Hessian. GPTQ performs sequential quantization, updating remaining weights using curvature information to compensate for quantization error. By capturing inter-weight dependencies, it enables accurate low bit (3–4 bit) quantization with moderate calibration data, though its handling of outliers remains implicit and depends on the quality of the curvature estimate.

1.3.3 AWQ: Activation-Aware Weight Quantization

AWQ (Lin et al., 2024) incorporates activation statistics to preserve weights critical to forward computation. Rather than minimizing weight error alone, it applies a per channel scaling s that minimizes output reconstruction after quantization:

$$s^* = \arg\min_s \Big\| Q\big(W \cdot \mathrm{diag}(s)\big) \cdot \big(\mathrm{diag}(s)^{-1} \cdot X\big) - WX \Big\|_2^2 \tag{4}$$

where $Q(\cdot)$ is the quantization operator. Salient weights are retained at higher precision, while the rest are aggressively quantized with per channel or group wise scaling. This reduces the effect of activation outliers and enables accurate 4 bit quantization without backpropagation or second order computations.

1.3.4 SmoothQuant

SmoothQuant (Xiao et al., 2023) is a post training quantization (PTQ) method that enables INT8 quantization for both weights and activations by suppressing activation outliers. It introduces a training free, mathematically equivalent transformation that migrates quantization difficulty from activations to weights. For a layer $Y = XW$, the transformation is:

$$\hat{X} = X\mathrm{diag}(s)^{-1}, \quad \hat{W} = \mathrm{diag}(s)W \qquad s_j = C_j^{\alpha}/W_j^{1-\alpha} \tag{5}$$

where C_j and W_j denote activation and weight channel scales. By selecting a migration strength α (typically 0.5), activation magnitudes are smoothed, enabling efficient INT8 execution with negligible accuracy loss while achieving up to $1.56\times$ speedup and $2\times$ memory reduction. However, the method is data-dependent, requiring a representative calibration set to estimate C_j, and its performance is sensitive to the choice of α, which must be tuned across model families to balance weight and activation quantization error.

1.4 Pruning

Pruning reduces model size by removing redundant parameters or structural components (e.g., attention heads, MLP neurons), altering architecture rather than precision. It includes unstructured pruning (removing individual weights, yielding sparsity) and structured pruning (removing entire units while preserving dense computation). In this work, we focus on Sheared LLaMA and SparseGPT.

1.4.1 Sheared LLaMA

Sheared LLaMA (Xia et al., 2024) employs targeted structured pruning to match a predefined architecture by removing layers, heads, and intermediate dimensions. The process is formulated as a constrained optimization problem using Lagrange multipliers; for a target number of heads H_T, the constraint is:

$$\overline{c}^{head}(\lambda,\phi,z) = \lambda^{head}\left(\sum z^{head} - H_{\mathcal{T}}\right) + \phi^{head}\left(\sum z^{head} - H_{\mathcal{T}}\right)^2 \tag{6}$$

Performance is recovered via dynamic batch loading, which adapts training data composition based on domain wise loss reduction. This approach is compute-efficient, requiring only $\sim$3% of the cost of training from scratch, and produces uniform architectures that are hardware friendly. However, the pruning stage itself is significantly slower (up to $5\times$ standard training), and performance recovery depends strongly on access to the original pre training data distribution, with missing domains leading to degraded knowledge retention.

1.4.2 SparseGPT

SparseGPT (Frantar and Alistarh, 2023) enables one-shot pruning of large scale models (100B+ parameters) without retraining by formulating pruning as a sparse regression problem. It leverages an approximate Optimal Brain Surgeon (OBS) update, where removing a weight w_m induces a compensation:

$$\delta_m = -\frac{w_m}{[H^{-1}]_{mm}} \cdot H^{-1}_{:,m}, \tag{7}$$

with H^{-1} denoting the inverse Hessian. By reusing Hessian information across rows, computational complexity is significantly reduced, enabling pruning of models such as OPT-175B within hours on a single GPU. The method supports both unstructured and semi-structured sparsity (e.g., 2:4) with minimal perplexity degradation. However, structured sparsity constraints (e.g., block or column pruning) introduce greater accuracy loss compared to unstructured pruning, while unstructured sparsity, although more accurate, requires specialized hardware or kernels to realize practical inference speedups due to unchanged dense dimensions.

1.5 Outlier Suppression

Activation outliers pose a major challenge for low bit quantization by skewing scaling factors and amplifying reconstruction error. GPTQ addresses this issue implicitly through its Hessian weighted objective, which down weights perturbations in directions that minimally affect the output, though it does not explicitly isolate extreme activations. In contrast, AWQ explicitly incorporates activation statistics, identifying high-impact weights, proportional to both weight magnitude and expected activation, and preserving them in higher precision to prevent dominant activations from distorting quantization scales. SmoothQuant takes a complementary approach by applying a pre conditioning transformation that migrates magnitude from activations to weights via channel wise rescaling thereby smoothing activation distributions and enabling more uniform quantization.

The aforementioned techniques are the most popular for outlier suppression. However, there also a few other recent techniques like QuIP# (Quantization with Incoherence Processing #), SpQR(Sparse-Quantized Representation), OmniQuant, and CushionCache. The QuIP# (Tseng et al., 2024) methodology multiplies weight and activation matrices by a random orthogonal matrix. This mathematically ensures that the resulting distribution is nearly Gaussian (incoherent) and free of extreme spikes. A recent review found that it achieved accuracy very close to baseline but required long GPU hours (Czakó et al., 2025). SpQR (Dettmers et al., 2023), on the other hand, tries to solve the problem by identifying the weights and activations that are most affected by quantization errors and keeps an insignificant fraction from it in the high precision FP16 format while quantizing all the remaining weights and activations. As a proposed improvement to SmoothQuant, OmniQuant (Shao et al., 2023) finds optimal channel wise scaling and shifting factors, and attention scaling factors using gradient descent to minimize the block wise reconstruction error, rather than relying on a heuristic. It also includes Learnable Weight Clipping (LWC) to find the best threshold for cutting off extreme weight values that might otherwise stretch the quantization grid too thin. This way it is able to give better results for across diverse model families. Finally, based on the concept of attention sinks and prefix-tuning, CushionCache (Son et al., 2024) utilizes greedy prefix search algorithm to iteratively select hard prompt tokens (tokens from the vocabulary) to initialize the prefix (Czakó et al., 2025). These are only a few of the many methodologies proposed in recent years.

2 Experiments

This section describes the experimental setup and execution of three quantization methods for **two LLM models** – OpenLLaMA 7B and OpenLLaMa 3B: Round-To-Nearest (RTN), GPTQ, and Activation-aware Weight Quantization (AWQ). All experiments were conducted using PyTorch on an NVIDIA A40-24Q GPU with 24 GB VRAM, running on a Linux system.

2.1 Calibration Data

Calibration data is essential for quantization methods like GPTQ and AWQ to optimize quantization parameters. We used the WikiText-2 dataset, a standard language modeling benchmark with diverse, high-quality text samples. A minimum of 128 calibration samples were loaded and tokenized for LLaMA 7B experiments and 64 for LLaMA 3B.

2.2 Implementation Details

RTN (Round-To-Nearest) implemented group wise scaling (default group size 128), per channel scaling, and support for 4 bit and 8 bit precision, requiring minimal computation and no calibration, making it the fastest method. GPTQ performs sequential, layer wise optimization using Hessian-based analysis and was implemented via the AutoGPTQ library. Its per layer optimization made it significantly slower than RTN. AWQ, implemented using AutoAWQ, preserves weights with high activation importance during calibration, achieving a balance of speed and accuracy, and demonstrating particular robustness at low bit width quantization.

3 Evaluation

This section outlines the evaluation methodology used to assess the performance of the quantized models compared to the baseline. The evaluation process measures model size, perplexity, and downstream task performance using standardized benchmarks.

3.1 Evaluation Metrics

Three primary metrics were used to evaluate the quantized models:

- **Model Size**: Calculated the compressed model size in GB compared to the FP16 baseline. This quantifies the memory and storage benefits of quantization.
- **Perplexity**: Measured on a subset of the WikiText-2 dataset (100 samples, maximum sequence length of 512 tokens). Perplexity indicates how well the model predicts the next token in a sequence, with lower values (lower confusion) indicating better performance.
- **LM Harness Tasks**: Evaluated on a selection of tasks from the Language Model Evaluation Harness (LM Eval), including HellaSwag, PIQA, MMLU STEM, and MMLU Humanities. Each task used 3-shot prompting with a limit of 20 samples per task to balance accuracy and computational efficiency.
- **Throughput and Latency**: Average inference speed for text generation and throughput (tokens processed per second) is measured for quantized models with reference to the baseline fp16 model.

4 Results

The quantization results for LLaMA-7B and LLaMA-3B are summarized in Tables 3 and 4. Visual illustrations are provided in Figures 1–3b and Figures 2–4b respectively. We also studied the sensitivity of MLP and attention layers to RTN quantization (Table 1) and observed that perplexity degrades more when MLP layers are quantized. As expected, reducing weight precision significantly decreases memory usage while introducing modest degradation in model quality. For both models, 8 bit RTN quantization achieves nearly identical perplexity and MMLU scores compared to the FP16 baseline while reducing weight memory by approximately 50%, indicating that 8 bit quantization preserves most of the representational capacity of the original model. Moving to 4 bit RTN further reduces memory consumption by half again, but results in noticeably higher perplexity, suggesting that uniform low bit quantization begins to limit the model's ability to represent fine-grained weight distributions.

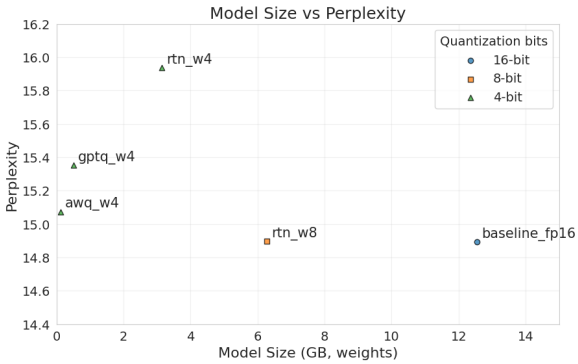


Figure 1: LLaMA-7B: Memory vs perplexity. The baseline fp16 model occupies about 13 GB memory with the least perplexity. RTN W8 halves the memory requirement while retaining accuracy. RTN W4 further halves the memory while compromising accuracy severely. AWQ W4 requires only about 0.1 GB memory while maintaining competitive perplexity.

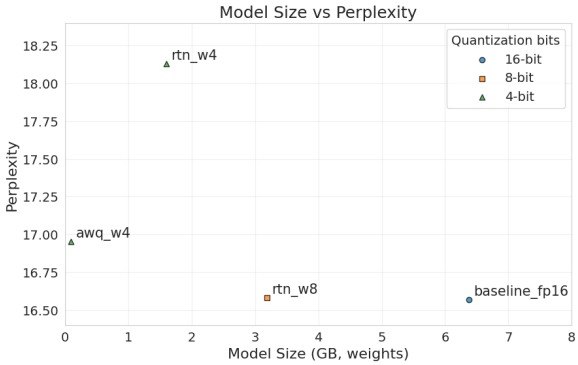
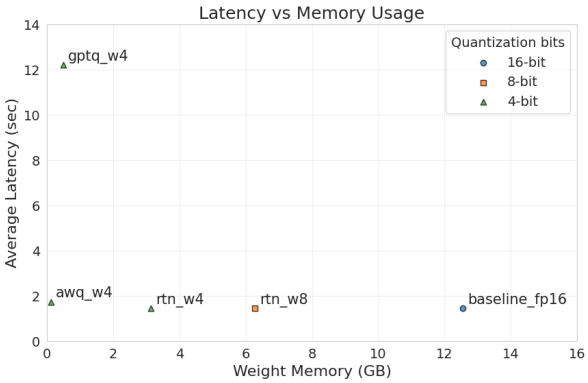
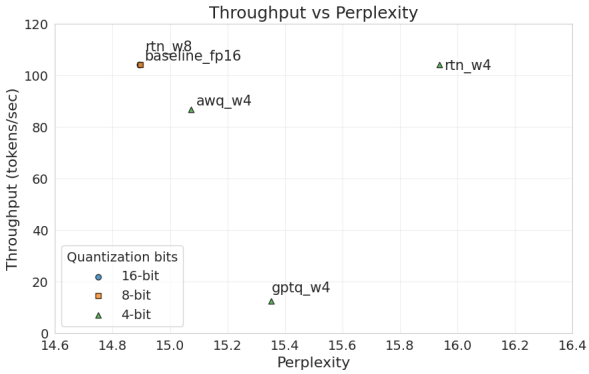


Figure 2: LLaMA-3B: Memory vs perplexity. Perplexity is higher for all models in comparison to LLaMA-7B, but at halved memory footprints.

More advanced 4 bit quantization methods such as AWQ and GPTQ demonstrate improved quality compared to naive RTN at the same bitwidth. In particular, AWQ achieves lower perplexity and higher MMLU scores than RTN 4bit for both models, because it selectively preserves important weights based on activation statistics during calibration. GPTQ also improves accuracy relative to RTN by minimizing quantization error through layer wise reconstruction, although its runtime performance in our setup is substantially slower, resulting in significantly higher latency and reduced token throughput.

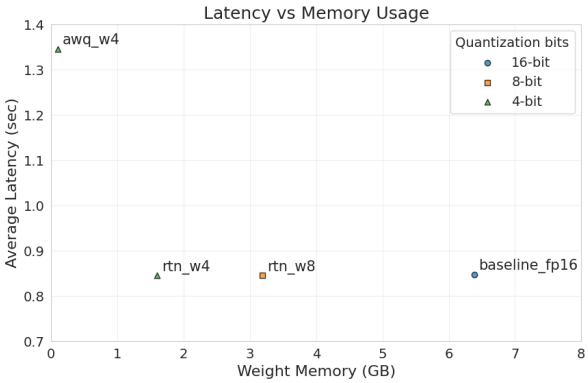


(a) Latency vs memory. All models exhibited similar inference latency, except GPTQ which took significantly longer because of the underlying implementation in AutoGPTQ library.

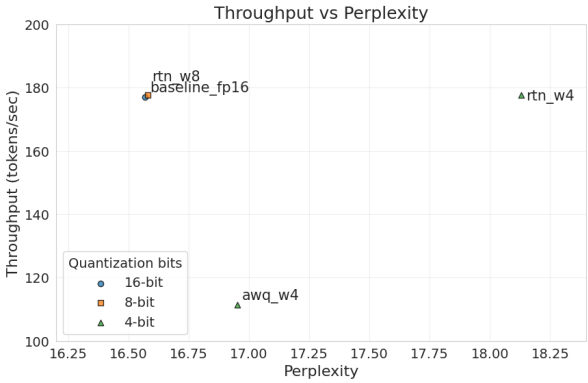


(b) Throughput vs perplexity. AWQ exhibits slightly lower throughput due to the additional runtime overhead required for group wise scaling and dequantization during inference.

Figure 3: LLaMA-7B: Inference efficiency trade-offs.



(a) Latency vs memory. The 3B models are almost twice as fast as 7B ones, except AWQ which is 23% faster.



(b) Throughput vs perplexity. Throughput for 3B models follows a similar trend as 7B. The 3B models process almost 1.7 times more tokens per second than 7B models.

Figure 4: LLaMA-3B: Inference efficiency trade-offs.

GPTQ models showed significantly higher inference latency, likely due to overhead from compressed weight formats requiring dequantization during matrix multiplications. Additionally, the AutoGPTQ implementation may rely on low bit kernels that are not fully optimized or supported on the current hardware, leading to slower fallbacks. Thus, while GPTQ improves accuracy over naive 4 bit RTN, it can introduce substantial inference overhead in practice (see Fig. 3a).

RTN quantized models show latency and throughput nearly identical to the FP16 baseline, indicating that the inference kernels in our setup do not significantly benefit from lower precision computation despite reduced memory usage. In contrast, AWQ and GPTQ incur additional overhead from specialized formats and dequantization, leading to higher latency. Overall, this reflects a trade-off between memory, accuracy, and efficiency: AWQ 4bit offers a strong balance of quality and compression, while RTN 8bit achieves near-lossless compression with minimal performance impact.

While the newer Llama 3 has fewer parameters than Llama 2 (7B), the quadrupling of the vocabulary size (128k in 3B vs 32k in 7B) introduces significant computational overhead in the final linear layers during Hessian-based quantization (GPTQ), leading to longer quantization wall-clock times on memory-constrained hardware. We believe that grouped-query attention used in Llama 3 might be unoptimized in AutoGPTQ, hence, we weren't able to run GPTQ on Llama-3B.

We also qualitatively compared model outputs across quantization levels using a set of common prompts (Table 2). The FP16 LLaMA-7B baseline produced coherent and diverse continuations across prompts, while heavily quantized models exhibited

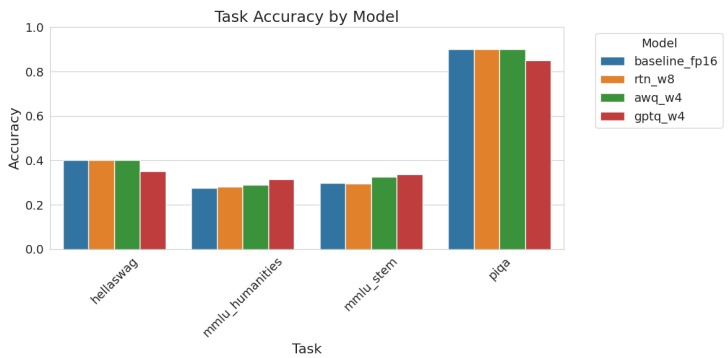


Figure 5: LLaMA-7B: Task accuracy across standardized LLM benchmarking tasks (HellaSwag, PIQA, and MMLU STEM). Quantized models remain broadly competitive with the baseline across tasks.

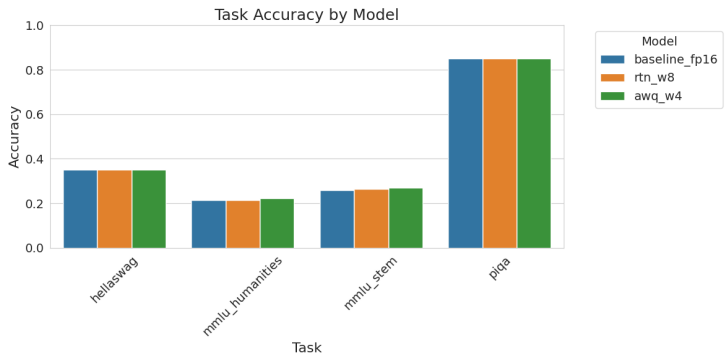


Figure 6: LLaMA-3B: Task accuracy across standardized LLM benchmarking tasks. Task accuracies are comparable for each quantized model with the baseline, however all values are lower than the 7B counterparts.

Table 1: Module Sensitivity to Quantization: Perplexity Comparison

Configuration	**Llama-3B Perplexity**	**Llama-7B Perplexity**
Baseline (FP16)	**12.6**	**11.1**
MLP Only	13.3	11.6
Attention Only	12.8	11.4
All Layers	13.8	12.2

Table 2: Qualitative comparison of generated responses from the best-performing model (LLaMA-7B FP16) and the most degraded model (LLaMA-3B RTN 4bit). The heavily quantized model exhibits repetitive generation patterns.

Prompt	**LLaMA-7B FP16 (Baseline)**	**LLaMA-3B RTN 4bit**
The weather today is	a bit of a mixed bag. It is slightly cloudy and windy, and a bit cold. I am not sure what the weather will be like tomorrow, but it looks like it may clear up later in the day.	a little bit of everything. It's been a little bit of everything. It's been a little bit of everything. It's been a little bit of everything.
The meaning of life is	to find your gift. The purpose of life is to give it away. The quote reflects the idea that meaning comes from discovering one's abilities and contributing them to others.	to be happy. The meaning of life is to be happy. The meaning of life is to be happy. The meaning of life is to be happy.

Table 3: Quantization evaluation for LLaMA-7B.

Model	Bits	Params	Weight Mem (GB)	Perplexity	MMLU Stem	Latency (s)	Throughput (tok/s)
baseline_fp16	16	6.74B	12.551	14.894	0.297	1.4419	104.03
rtn_w8	8	6.74B	6.276	**14.896**	**0.295**	1.4425	103.98
rtn_w4	4	6.74B	3.138	15.937	0.313	**1.4419**	104.03
awq_w4	4	0.26B	0.122	15.074	0.326	1.7306	86.68
gptq_w4	4	1.07B	**0.500**	15.353	0.337	12.2019	12.29

Table 4: Quantization evaluation for LLaMA-3B.

Model	Bits	Params	Weight Mem (GB)	Perplexity	MMLU Stem	Latency (s)	Throughput (tok/s)
baseline_fp16	16	3.43B	6.382	16.568	0.258	0.8470	177.09
rtn_w8	8	3.43B	3.191	**16.579**	0.263	**0.8445**	177.61
rtn_w4	4	3.43B	1.596	18.130	0.289	0.8448	177.57
awq_w4	4	0.20B	**0.095**	16.952	0.271	1.3456	111.47

noticeable degradation in generation quality. In particular, the 3B RTN 4bit model frequently produced repetitive text loops, a known failure mode of aggressively quantized language models. This behavior likely arises from increased quantization noise distorting token probability distributions during decoding. Despite this degradation in fluency, the quantized models generally retained core factual knowledge (e.g., correctly identifying Paris as the capital of France, in another example not shown in the table). These qualitative observations are consistent with the quantitative metrics in Tables 3 and 4, where lower bit RTN models exhibit higher perplexity and lower MMLU scores.

Acknowledgments

We thank Prof. Farinaz Koushanfar, our ECE 226 instructor, for giving us the opportunity to conduct this study.

Contributions

Note: While we divided survey and hardware experiment tasks due to time constraints, we ensured that we discussed our individual findings and inferences with other teammates through weekly project meetings.

Kishore worked on the quantization schemes, developing code, designing the experiments for evaluation of quantized LLMs and inferring the underlying results.
Ayush worked on developing code, running quantization and evaluation experiments on hardware, reporting and inferring the results thus obtained.
Arnav contributed to the literature survey, collecting relevant articles, summarizing the structural pruning and latest outlier suppression techniques, and developing the presentation deck.

References

Patrik Czakó, Gábor Kertész, and Sándor Szénási. 2025. Addressing activation outliers in llms: A systematic review of post-training quantization techniques. *IEEE Access*.

Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. 2023. Spqr: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078*.

Elias Frantar and Dan Alistarh. 2023. Sparsegpt: massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. Gptq: Accurate post-training quantization for generative pre-trained transformers. *Preprint*, arXiv:2210.17323.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for llm compression and acceleration. In *MLSys*.

Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2023. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*.

Seungwoo Son, Wonpyo Park, Woohyun Han, Kyuyeun Kim, and Jaeho Lee. 2024. Prefixing attention sinks can mitigate activation outliers for large language model quantization. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 2242–2252.

Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. 2024. Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks. *Proceedings of machine learning research*, 235:48630.

Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2024. Sheared llama: Accelerating language model pre-training via structured pruning. *Preprint*, arXiv:2310.06694.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org.